

Contents

□ Smart Samarpan Academy — Complete Interview Questions Bank	1
A. PROJECT INTRODUCTION — 10 Questions	1
B. FEATURES — 15 Questions	2
C. TECH STACK — 15 Questions	5
D. CODE — 20 Questions	7
E. ARCHITECTURE — 15 Questions	9
F. DATABASE — 15 Questions	11
G. APIs — 15 Questions	13
H. CHALLENGES — 10 Questions	15
I. FUTURE IMPROVEMENTS — 10 Questions	16
J. TRICKY QUESTIONS — 20 Questions	17

□ **Smart Samarpan Academy — Complete Interview Questions Bank**

145+ questions based ONLY on your actual codebase. Nothing invented. Every question has:
Question → **What interviewer wants** → **Short answer** → **Deep explanation** → **Follow-up**

A. PROJECT INTRODUCTION — 10 Questions

Q1. Tell me about your project.

- **What interviewer wants:** Can you explain clearly? Do you understand what you built?
- **Short answer:** “I built Samarpan Math Academy — a full-stack AI-powered math learning platform for Class 9-12 students. Students can buy courses via Razorpay, watch video lectures, take AI-generated quizzes using Google Gemini, and track their progress. There’s also an admin dashboard for managing courses, lectures, and users.”
- **Deep:** Mention MERN-ish stack (React + Vite + Tailwind, Node.js + Express, MongoDB), 3 AI features (quiz, formula, performance analysis), payment integration, OTP email verification, Google OAuth.
- **Follow-up:** “What makes your project different from a simple course platform?”

Q2. What problem does your project solve?

- **Wants:** Can you articulate the value proposition?
- **Short:** “Traditional math learning lacks personalized practice. My platform uses AI to generate custom quizzes tailored to any topic and difficulty, and tracks learning progress lecture by lecture.”
- **Deep:** 4 problems: no personalized quizzes, no instant formula reference, no progress tracking, no affordable online math-specific platform for Indian students.
- **Follow-up:** “Who are your target users?”

Q3. Who are the target users?

- **Short:** “Three types — students (Class 9-12), teachers/admins who create content, and parents who buy courses.”
- **Follow-up:** “How do you handle different user types in code?” → Role-based system (user/admin/superadmin).

Q4. What is the tech stack?

- **Wants:** Can you list and justify each technology?

- **Short:** “React with Vite for frontend, Tailwind CSS for styling, Node.js with Express for backend, MongoDB with Mongoose for database, Google Gemini for AI features, Razorpay for payments. Deployed on Vercel (frontend) and Render (backend).”
- **Follow-up:** “Why Vite over Create React App?”

Q5. How many APIs does your project have?

- **Short:** “Around 27 REST API endpoints covering authentication (6 endpoints), course management (7), admin operations (7), and AI features (4), plus payment (2).”
- **Follow-up:** “Which API was hardest to implement?”

Q6. Did you work on this alone or in a team?

- **Short:** Answer honestly based on your situation.
- **Follow-up:** “What was the most time-consuming part?”

Q7. How long did it take to build?

- **Short:** Answer honestly.
- **Follow-up:** “If you had to rebuild it, what would you do differently?”

Q8. What is the most unique feature?

- **Short:** “The AI quiz generation — students type any math topic, select difficulty (Olympiad level, JEE level, or school level), and get instant MCQs generated by Google Gemini with KaTeX-rendered math formulas.”
- **Follow-up:** “How do you ensure the AI returns valid quiz data?”

Q9. Is the project deployed?

- **Short:** “Yes — frontend on Vercel at samarpan-math-academy.vercel.app, backend on Render at samarpan-guzg.onrender.com, database on MongoDB Atlas.”
- **Follow-up:** “What deployment issues did you face?”

Q10. Walk me through the user journey.

- **Short:** “User registers with email OTP → Logs in → Browses courses → Buys via Razorpay → Watches video lectures → Marks lectures complete → Takes AI quizzes → Views performance analysis → Tracks progress.”
- **Follow-up:** “What happens if the user isn’t logged in and tries to access a course?”

B. FEATURES — 15 Questions

Q11. How does user registration work?

- **Wants:** Understanding of the OTP-based registration flow.
- **Short:** “User fills form → backend hashes password with bcrypt → generates 6-digit OTP → embeds user data + OTP in a JWT (5-minute expiry) → sends OTP to email via Nodemailer → user enters OTP → backend verifies JWT, compares OTP, creates user in MongoDB.”
- **Deep:** OTP is NOT stored in database — it lives inside the JWT payload. This is a stateless OTP approach.
- **Follow-up:** “Why didn’t you store the OTP in the database?”

Q12. How does the AI quiz generation work?

- **Wants:** Understanding of external API integration.
- **Short:** "User selects topic and difficulty → frontend sends POST /api/generate-quiz → backend builds a detailed prompt → calls Gemini API with fetchWithRetry → strips markdown code fences from response → parses JSON → returns quiz questions array."
- **Deep:** Difficulty maps to prompt text: '9th_10th_Olympiad' → 'Olympiad level', '11th_12th_JEE' → 'JEE level'. Uses fetchWithRetry for rate limit handling.
- **Follow-up:** "What if Gemini returns invalid JSON?"

Q13. How does payment work?

- **Wants:** Understanding of payment gateway integration and security.
- **Short:** "Two-step: (1) Backend creates Razorpay order with amount in paise (price × 100). (2) Frontend opens Razorpay checkout. On success, frontend sends order_id, payment_id, signature to backend. Backend creates HMAC-SHA256 hash and compares with Razorpay's signature. If matches, saves Payment, enrolls user, creates Progress tracker."
- **Follow-up:** "What happens if someone sends a fake payment success to your backend?"

Q14. How does progress tracking work?

- **Wants:** Understanding of data modeling.
- **Short:** "A Progress document is created when user buys a course (completedLectures: empty array). When user watches a lecture and clicks 'Mark Complete', backend checks if lectureId is already in the array (idempotency). If not, pushes it. Frontend shows percentage: completedLectures.length / totalLectures × 100."
- **Follow-up:** "What if the user clicks 'Mark Complete' twice?"

Q15. How does the admin dashboard work?

- **Wants:** Role-based access understanding.
- **Short:** "Admin routes are protected by isAuthenticated + isAdmin middleware chain. Admin can CRUD courses (with image upload), CRUD lectures (with video upload), view stats (total users/courses/lectures), manage users (change roles, delete)."
- **Follow-up:** "Can any admin promote another user to admin?"

Q16. How does Google OAuth login work?

- **Wants:** Understanding of OAuth2.
- **Short:** "Frontend uses @react-oauth/google to get a Google ID token. Sends to POST /api/user/google-login. Backend verifies with google-auth-library's verifyIdToken. If user exists, logs in. If not, creates new user with a dummy hashed password and logs in."
- **Follow-up:** "What's the dummy password for? Can Google users reset their password?"

Q17. How does the formula generator work?

- **Short:** "User enters topic, class level, and competitive level. Backend sends a detailed prompt to Gemini asking for formulas in JSON format with LaTeX. Frontend renders formulas using KaTeX library."
- **Follow-up:** "Why KaTeX over MathJax?"

Q18. How is the search/filter implemented on the courses page?

- **Short:** “Frontend-side filtering using JavaScript. Users can search by query (name match), filter by category (Class 9/10/11/12), and toggle grid/list view. No server-side search.”
- **Follow-up:** “Would server-side search be better? When?”

Q19. How do you handle file uploads?

- **Short:** “Using Multer middleware with diskStorage. Files saved to uploads/ directory with UUID-generated filenames. Two upload handlers: uploadImage (for course images) and uploadVideo (for lecture videos). Files served as static via Express.”
- **Follow-up:** “What are the limitations of local file storage?”

Q20. How does the performance analysis feature work?

- **Short:** “Backend fetches all quiz results for the user, formats them into a prompt, sends to Gemini API asking for: textReport (markdown analysis), topicPerformance (bar chart data), overallStats (pie chart data). Frontend renders with Recharts.”
- **Follow-up:** “What if the user has no quiz history?”

Q21. How does the forgot password flow work?

- **Short:** “User enters email → backend generates JWT with email (no expiry defined, but sets resetPasswordExpire on user to 5 minutes) → sends email with reset link containing token → user clicks link → enters new password → backend verifies token, checks expiry, hashes new password, saves.”
- **Follow-up:** “What if the token is stolen?”

Q22. How does course deletion work for admins?

- **Short:** “It’s a cascading delete: (1) Delete all lecture video files from disk using Promise.all + fs.unlink, (2) Delete course image from disk, (3) Delete all lecture documents from MongoDB, (4) Delete course document, (5) Remove courseId from all users’ subscription arrays using User.updateMany with \$pull.”
- **Follow-up:** “What if a file deletion fails?”

Q23. What happens after a successful payment?

- **Short:** “Three things in sequence: (1) Payment document saved, (2) Course ID pushed to user’s subscription array, (3) Progress document created with empty completedLectures. User is then enrolled and can access lectures.”
- **Follow-up:** “Can a user buy the same course twice?”

Q24. How do you prevent unauthorized access to premium lectures?

- **Short:** “Server-side check in fetchLectures and fetchLecture: checks if user.subscription.includes(courseId). Admins bypass this check with role === ‘admin’. Even if frontend hides the content, backend enforces access control.”
- **Follow-up:** “Is this enough? What about the video file URL?”

Q25. What AI features does your project have?

- **Short:** “Three: (1) Quiz Generation — custom MCQs on any topic, (2) Formula Generator — LaTeX formulas for any chapter, (3) Performance Analysis — AI analyzes quiz history and provides study recommendations with chart data.”
- **Follow-up:** “Which AI model do you use and why?”

C. TECH STACK — 15 Questions

Q26. Why React? Why not Angular or Vue?

- **Short:** “React’s component-based architecture and huge ecosystem made it ideal. Context API was sufficient for state management at this scale. Angular would be overkill with its opinionated structure. Vue is good but has a smaller job market in India.”
- **Follow-up:** “Did you consider using Next.js?”

Q27. Why Vite over Create React App?

- **Short:** “CRA is deprecated and uses Webpack which is slow. Vite uses esbuild for pre-bundling — dev server starts in milliseconds, HMR is near-instant. It’s the modern standard.”
- **Follow-up:** “What does HMR mean?”

Q28. Why MongoDB over PostgreSQL?

- **Short:** “Flexible schema fits evolving requirements — quiz results have dynamic structures. JSON-native storage works naturally with Node.js. No complex joins needed — I use Mongoose references and populate. MongoDB Atlas provides easy cloud hosting.”
- **Follow-up:** “When would PostgreSQL be better?”

Q29. Why Express.js specifically?

- **Short:** “Most minimal and popular Node.js framework. Great middleware ecosystem. Easy to learn. Perfect for REST APIs. Express 5 is used which has built-in async error handling.”
- **Follow-up:** “What alternatives exist? Why not Fastify?”

Q30. Why Tailwind CSS?

- **Short:** “Utility-first approach means faster development — no switching between CSS files. Responsive design built-in. Smaller production bundle because unused styles are purged. More customizable than Bootstrap.”
- **Follow-up:** “What’s the disadvantage?”

Q31. Why JWT over session-based auth?

- **Short:** “Stateless — server doesn’t need to store sessions. Works across domains (frontend on Vercel, backend on Render). Scalable — no shared session store needed for load balancing. Contains user ID in payload.”
- **Follow-up:** “What are JWT’s disadvantages?”

Q32. Why bcrypt for password hashing?

- **Short:** “bcrypt is slow by design — makes brute-force attacks computationally expensive. Adds a random salt automatically, preventing rainbow table attacks. Industry standard with salt rounds parameter (I use 10).”
- **Follow-up:** “What’s the difference between encryption and hashing?”

Q33. Why Razorpay over Stripe?

- **Short:** “Razorpay is optimized for the Indian market — supports UPI, Indian cards, netbanking. Has better documentation for INR transactions. Stripe has limited UPI support.”
- **Follow-up:** “How does Razorpay’s signature verification work?”

Q34. Why Google Gemini API over OpenAI?

- **Short:** “Gemini has a free tier which is great for student projects. The Flash model is fast. It handles structured JSON output reasonably well for quiz generation.”
- **Follow-up:** “What model specifically do you use?” → gemini-flash-latest

Q35. Why Mongoose over raw MongoDB driver?

- **Short:** “Mongoose provides schema validation, type casting, query building, and middleware (pre/post hooks). Makes MongoDB feel structured while keeping flexibility.”
- **Follow-up:** “What’s a Mongoose middleware?”

Q36. Why Axios over fetch API?

- **Short:** “Axios has cleaner syntax, automatic JSON transformation, request/response interceptors, better error handling (throws on non-2xx status), and timeout support.”
- **Follow-up:** “How do you add the auth token to requests?”

Q37. Why Multer for file uploads?

- **Short:** “Multer is the standard Express middleware for multipart/form-data parsing. Simple API — configure storage destination and filename, then use as middleware.”
- **Follow-up:** “What’s multipart/form-data?”

Q38. Why Nodemailer?

- **Short:** “Most popular Node.js email library. Supports SMTP (Gmail), has HTML template support. Easy to configure with Gmail app passwords.”
- **Follow-up:** “Why does it fail on Render?”

Q39. Why KaTeX over MathJax?

- **Short:** “KaTeX renders faster — it’s designed for speed. Lighter bundle size. For a quiz page rendering multiple math expressions, speed matters.”
- **Follow-up:** “What is LaTeX?”

Q40. Why React Context API over Redux?

- **Short:** “Project has only 2 contexts (UserContext, CourseContext) with simple state. Redux would be overkill. Context API is built into React — no extra dependency. For this scale, performance is not an issue.”
 - **Follow-up:** “When would you need Redux?”
-

D. CODE — 20 Questions

Q41. Explain how `isAuth` middleware works line by line.

- **Wants:** Deep code understanding.
- **Short:** “Extracts token from `req.headers.token` → verifies with `jwt.verify` using `JWT_SECRET` → finds user by `decoded._id` → attaches to `req.user` → calls `next()`. If any step fails, returns 401.”
- **Follow-up:** “Why not use `Authorization: Bearer` header?”

Q42. What does the `TryCatch` middleware do?

- **Short:** “It’s a higher-order function that wraps controller functions in `try/catch`. Returns a new function that calls the original inside `try`, and calls `next(error)` on `catch`. Prevents writing `try/catch` in every controller.”
- **Follow-up:** “What’s a higher-order function?”

Q43. Explain the `fetchWithRetry` function.

- **Short:** “Calls `fetch`. If response is 429 (rate limit), waits for `backoffMs` (starts at 2 seconds), then retries. Doubles the wait time each retry. Max 3 retries. If still 429 after 3 retries, returns the 429 response.”
- **Follow-up:** “What is exponential backoff?”

Q44. How do you clean AI response before parsing?

- **Short:** “`quizJsonString.replace(/```json\n|```/g, '').trim()` — removes markdown code fences that Gemini sometimes wraps around JSON responses. Then `JSON.parse` on the cleaned string.”
- **Follow-up:** “What if the AI returns completely invalid JSON?”

Q45. How is OTP generated?

- **Short:** “`Math.floor(Math.random() * 1000000)` — generates a number between 0 and 999999. This is a 6-digit OTP.”
- **Deep:** This is not cryptographically secure. For production, use `crypto.randomInt(100000, 999999)`.
- **Follow-up:** “Is `Math.random()` secure enough for OTPs?”

Q46. How does `bcrypt.hash` work?

- **Short:** “`await bcrypt.hash(password, 10)` — the 10 is salt rounds. Higher = more secure but slower. `bcrypt` generates a random salt, prepends it to the password, and hashes it. The salt is stored within the hash string.”
- **Follow-up:** “What are salt rounds?”

Q47. How does `jwt.sign` work in your `register` function?

- **Short:** “`jwt.sign({user, otp}, Activation_Secret, {expiresIn: '5m'})` — creates a token containing user data and OTP, signed with a secret key, auto-expires in 5 minutes.”
- **Follow-up:** “What happens if someone uses an expired token?”

Q48. How does payment signature verification work?

- **Short:** “`crypto.createHmac('sha256', Razorpay_Secret).update(order_id + '|' + payment_id).digest('hex')` — creates a hash using HMAC-SHA256 algorithm. Compares this hash with Razorpay’s signature. If they match, payment is authentic.”
- **Follow-up:** “Why HMAC-SHA256? Why not MD5?”

Q49. How does the subscription check work in `fetchLectures`?

- **Short:** “`user.subscription.includes(req.params.id)` — checks if the course ID exists in the user’s subscription array. If not, returns 400. Admin bypasses this with `user.role === 'admin'.`”
- **Follow-up:** “What’s the time complexity of `includes()`?”

Q50. How does `deleteCourse` handle cleanup?

- **Short:** “Uses `Promise.all` to concurrently unlink all lecture videos, then `rm`’s the course image, deletes all lecture documents, deletes the course, and uses `User.updateMany` with `$pull` to remove the course from all users’ subscriptions.”
- **Follow-up:** “What is `$pull` in MongoDB?”

Q51. How does `addProgress` ensure idempotency?

- **Short:** “Checks `progress.completedLectures.includes(lectureId)` before pushing. If already exists, returns ‘Progress recorded’ without adding again.”
- **Follow-up:** “What is idempotency?”

Q52. How does `getYourProgress` calculate percentage?

- **Short:** “Counts `completedLectures.length` from `Progress`, counts total lectures from `Lecture.find({course})`, divides: $(completedLectures * 100) / allLectures.$ ”
- **Follow-up:** “What if `allLectures` is 0?” → Division by zero!

Q53. How does `updateRole` work?

- **Short:** “First checks `req.user.mainrole === 'superadmin'.` If not superadmin, returns 403. Then toggles the target user’s role between ‘user’ and ‘admin’.”
- **Follow-up:** “What’s the difference between role and mainrole?”

Q54. How does Google OAuth token verification work on the backend?

- **Short:** “`client.verifyIdToken({idToken: token, audience: GOOGLE_CLIENT_ID})` — Google’s library validates the token was issued by Google, for your app, and hasn’t expired. Returns payload with email, name, picture.”
- **Follow-up:** “What is audience in OAuth?”

Q55. How does the Multer configuration work?

- **Short:** “`diskStorage` with destination: ‘uploads/’, filename: `uuid() + originalExtension.` Creates `uploadFiles` middleware that handles single file upload field. Used for both images and videos.”
- **Follow-up:** “Why UUID for filenames?”

Q56. How does sendMail work?

- **Short:** "Creates Nodemailer transport with smtp.gmail.com:465 (SSL). Constructs HTML email template with OTP. Sends via transport.sendMail(). Uses Gmail app password for auth."
- **Follow-up:** "What's an app password?"

Q57. How is the Gemini API prompt structured for quiz generation?

- **Short:** "Builds a prompt asking for N-question MCQ quiz on topic X at difficulty Y. Specifies exact JSON format with question, options array, correct_answer. Sends as chatHistory array with user role."
- **Follow-up:** "What is prompt engineering?"

Q58. How does the frontend store the auth token?

- **Short:** "After login, token is saved to localStorage. On every API request, it's sent via headers. On page load, fetchUser checks localStorage for token and validates with GET /api/user/me."
- **Follow-up:** "Is localStorage secure for tokens?"

Q59. How do you handle CORS?

- **Short:** "Using cors middleware with origin callback that currently allows all origins. For production, should validate against allowedOrigins array."
- **Follow-up:** "What is CORS and why is it needed?"

Q60. How does the React Router protect routes?

- **Short:** "ProtectedRoute component checks isAuthenticated from context. If not authenticated, redirects to /login. For admin routes, also checks user.role === 'admin'. Shows Loading while auth state is being determined."
 - **Follow-up:** "Is frontend protection enough?"
-

E. ARCHITECTURE — 15 Questions

Q61. Explain the overall architecture of your project.

- **Short:** "3-tier architecture: React frontend (Vercel) → Express REST API (Render) → MongoDB (Atlas). Plus external services: Gemini AI, Razorpay, Google OAuth, Gmail SMTP."
- **Follow-up:** "Draw the data flow diagram."

Q62. Why did you separate frontend and backend?

- **Short:** "Independent deployment — frontend on Vercel's CDN for speed, backend on Render for compute. Can scale each independently. Enables different teams to work on each. API can serve mobile apps too."
- **Follow-up:** "How do they communicate?"

Q63. How does the middleware chain work?

- **Short:** "Request flows: CORS → express.json → Route → isAuthenticated (if protected) → isAdmin (if admin route) → multer (if file upload) → Controller → Response."
- **Follow-up:** "What does next() do?"

Q64. What is the MVC pattern and does your project follow it?

- **Short:** “Model-View-Controller. My project follows it: Models (Mongoose schemas in models/), Views (React components), Controllers (controller functions). Routes map URLs to controllers.”
- **Follow-up:** “Where is the business logic?”

Q65. How do you handle errors across the application?

- **Short:** “Backend uses TryCatch higher-order function to wrap controllers. Catches errors and returns 500 with message. Frontend uses try-catch in context functions and displays errors via react-hot-toast.”
- **Follow-up:** “Do you have global error handling middleware?”

Q66. Explain the authentication flow.

- **Short:** “Register: OTP in JWT → Verify → User created. Login: bcrypt compare → JWT issued (15 days). Every protected request: token in header → isAuthenticated verifies → controller executes.”
- **Follow-up:** “What happens when the token expires?”

Q67. How do you manage state in the frontend?

- **Short:** “Two React Contexts: UserContext (auth state, user data, login/register/logout functions) and CourseContext (courses list, single course, my courses). No Redux needed at this scale.”
- **Follow-up:** “How does Context API work internally?”

Q68. Where does business logic live?

- **Short:** “In three places: Controllers (main logic), Middlewares (auth, file upload, email), and some inline in index.js (AI features, payment). Ideally, inline routes should be moved to controllers.”
- **Follow-up:** “Would you refactor the inline routes?”

Q69. How does the file serving work?

- **Short:** “`app.use('/uploads', express.static('uploads'))` — Express serves the uploads directory as static files. Frontend accesses images/videos via `http://backend-url/uploads/filename`.”
- **Follow-up:** “What’s the problem with this approach in production?”

Q70. How does the frontend know the backend URL?

- **Short:** “Currently, the server URL is exported from main.jsx (hardcoded). For production, it should use `VITE_API_URL` environment variable via `import.meta.env`.”
- **Follow-up:** “Why use environment variables?”

Q71. How does routing work in your SPA?

- **Short:** “React Router DOM handles client-side routing. `vercel.json` rewrites all routes to `index.html` so Vercel doesn’t return 404 on direct URL access.”
- **Follow-up:** “What’s the difference between client-side and server-side routing?”

Q72. How many database models do you have and how are they related?

- **Short:** “6 models: User, Courses, Lecture, Payment, Progress, QuizResult. User → Courses (subscription array). Courses → Lectures (one-to-many). User → Progress (one-to-many). User → QuizResult (one-to-many).”
- **Follow-up:** “Why not embed lectures inside courses?”

Q73. How does the Razorpay integration architecture work?

- **Short:** “3-step: (1) Backend creates order via Razorpay SDK, (2) Frontend loads Razorpay checkout script and opens modal, (3) Backend verifies payment cryptographically. Never trust the client for payment status.”
- **Follow-up:** “What if the verification API call fails after payment?”

Q74. What are the external services your app depends on?

- **Short:** “4 external services: MongoDB Atlas (database), Google Gemini API (AI), Razorpay (payments), Gmail SMTP (emails). Plus Google OAuth for login.”
- **Follow-up:** “What happens if Gemini goes down?”

Q75. Explain the admin access control architecture.

- **Short:** “Three levels: user → admin → superadmin. isAuth checks JWT. isAdmin checks role === ‘admin’. updateRole checks mainrole === ‘superadmin’. Only superadmin can promote/demote roles.”
 - **Follow-up:** “How is mainrole set?”
-

F. DATABASE — 15 Questions

Q76. What database do you use and why?

- **Short:** “MongoDB with Mongoose ODM. Chosen because of flexible schema (AI quiz results have dynamic structures), JSON-native storage works well with Node.js, easy horizontal scaling, MongoDB Atlas for managed hosting.”
- **Follow-up:** “What are MongoDB’s limitations?”

Q77. List all your collections and their purposes.

- **Short:** “6 collections: users (auth + subscriptions), courses (course catalog), lectures (video content linked to courses), payments (Razorpay transaction records), progresses (lecture completion tracking), quizresults (AI quiz scores).”
- **Follow-up:** “Which collection grows the fastest?”

Q78. Explain the User schema.

- **Short:** “name, email (unique), password (hashed), role (‘user’/‘admin’), mainrole (‘user’/‘superadmin’), subscription (array of Course ObjectIds), resetPasswordExpire (Date), timestamps.”
- **Follow-up:** “Why two role fields?”

Q79. How are courses and lectures related?

- **Short:** “One-to-many. Each Lecture document has a course field (ObjectId ref to Courses). To get all lectures for a course: `Lecture.find({course: courseId})`.”
- **Follow-up:** “Why not embed lectures inside the course document?”

Q80. How does the subscription/enrollment system work in the database?

- **Short:** “User has a subscription array of Course ObjectIds. When payment is verified, the course ID is pushed to this array. To check access: `user.subscription.includes(courseId)`.”
- **Follow-up:** “What if the array gets very large?”

Q81. What is the Progress collection and how is it structured?

- **Short:** “Tracks course completion per user. Fields: course (ref), user (ref), completedLectures (array of Lecture ObjectIds). Created when user buys a course. Updated when user marks lectures complete.”
- **Follow-up:** “How do you calculate completion percentage?”

Q82. Explain the QuizResult schema.

- **Short:** “user (ObjectId ref), topic (String), difficulty (enum: 9th_10th_Olympiad, 11th_12th_JEE, medium, easy, hard, Standard), score (Number), totalQuestions (Number), timestamps.”
- **Follow-up:** “Why is difficulty an enum?”

Q83. How is payment data stored?

- **Short:** “Payment collection stores: razorpay_order_id, razorpay_payment_id, razorpay_signature. Acts as an audit trail. No user reference stored directly — the user’s enrollment is tracked via their subscription array.”
- **Follow-up:** “Should you store the userId in the Payment document?”

Q84. How does data insertion happen for user registration?

- **Short:** “Two-step: (1) During register, user data is NOT inserted — it’s stored in JWT. (2) During verify, `User.create({name, email, password})` inserts the document.”
- **Follow-up:** “Why this two-step approach?”

Q85. How do you query the database for admin stats?

- **Short:** “`getAllStats: (await Courses.find()).length, (await Lecture.find()).length, (await User.find()).length`. This fetches ALL documents just to count — should use `countDocuments()`.”
- **Follow-up:** “Why is this inefficient?”

Q86. What are ObjectId references in your project?

- **Short:** “MongoDB ObjectIds link documents across collections. Like foreign keys. Example: Lecture has `course: ObjectId` pointing to a Courses document. Mongoose’s `populate()` can resolve these references.”
- **Follow-up:** “Do you use `populate()` anywhere?”

Q87. How does the \$pull operator work in your project?

- **Short:** “In `deleteCourse: User.updateMany({}, { $pull: { subscription: req.params.id } })` — removes the deleted course ID from ALL users’ subscription arrays in one query.”
- **Follow-up:** “What’s `updateMany`?”

Q88. What indexes exist in your database?

- **Short:** “By default, MongoDB creates an index on `_id`. The User schema has `email: unique: true` which creates a unique index on email. No other custom indexes are defined.”
- **Follow-up:** “What queries would benefit from indexes?”

Q89. How do timestamps work in Mongoose?

- **Short:** “Setting `timestamps: true` in schema options auto-adds `createdAt` and `updatedAt` fields. Managed by Mongoose on create/update operations.”
- **Follow-up:** “Which models use timestamps?”

Q90. What would you change about the database design?

- **Short:** “Add `userId` to Payment for audit. Use `countDocuments()` for stats. Add an `authProvider` field to User. Add indexes on frequently queried fields. Consider pagination for large collections.”
 - **Follow-up:** “How would you implement pagination?”
-

G. APIs — 15 Questions

Q91. What HTTP methods do you use and for what?

- **Short:** “GET for fetching data (courses, lectures, profile, stats, progress). POST for creating data (register, login, checkout, quiz generation). PUT for updating (role change). DELETE for removing (courses, lectures, users).”
- **Follow-up:** “Difference between PUT and PATCH?”

Q92. How do you send the auth token from frontend to backend?

- **Short:** “Via custom header: `headers: { token: localStorage.getItem('token') }`. Backend extracts with `req.headers.token`.”
- **Follow-up:** “Why not use Authorization: Bearer header?” → It’s non-standard but works.

Q93. What’s the difference between authenticated and public APIs?

- **Short:** “Public: GET `/api/course/all` (no middleware). Authenticated: GET `/api/user/me` (`isAuth` middleware). Admin: POST `/api/course/new` (`isAuth` + `isAdmin`).”
- **Follow-up:** “List all public APIs.”

Q94. How do you handle API errors?

- **Short:** “TryCatch wrapper catches exceptions. Controllers return appropriate status codes: 400 (bad request), 401 (unauthorized), 403 (forbidden), 404 (not found), 500 (server error). Frontend shows errors via toast.”
- **Follow-up:** “Do all your APIs have consistent error response format?”

Q95. Explain the checkout API flow.

- **Short:** “POST `/api/course/checkout/:id` — Checks user isn’t already subscribed. Creates Razorpay order with amount (`price × 100` for paise), currency INR. Returns order object and course details to frontend.”
- **Follow-up:** “Why multiply by 100?”

Q96. How does the quiz generation API differ from a normal CRUD API?

- **Short:** "It's a 'proxy' API — receives request from frontend, transforms it into a Gemini API call, processes the AI response, and returns cleaned data. No database read/write (until quiz submission)."
- **Follow-up:** "What's the latency of this API?"

Q97. Which APIs use file upload?

- **Short:** "POST /api/course/new (course image), POST /api/course/:id (lecture video). Both use Multer uploadFiles middleware."
- **Follow-up:** "What's the max file size?" → Not configured in the code (Multer default).

Q98. How do query parameters work in your APIs?

- **Short:** "addProgress uses query params: POST /api/user/progress?course=X&lectureId=Y. resetPassword uses: POST /api/user/reset?token=X. Accessed via req.query."
- **Follow-up:** "When to use query params vs request body?"

Q99. What status codes do you return?

- **Short:** "200 (success), 201 (created), 400 (bad request), 401 (unauthorized), 403 (forbidden), 404 (not found), 429 (rate limit from Gemini), 500 (server error)."
- **Follow-up:** "What's the difference between 401 and 403?"

Q100. How would you document your APIs?

- **Short:** "I would use Swagger/OpenAPI specification with swagger-jsdoc and swagger-ui-express to auto-generate API documentation."
- **Follow-up:** "Have you used Postman for testing?"

Q101. How many APIs are inline in index.js vs in separate route files?

- **Short:** "4 inline (generate-quiz, generate-formulas, submit-quiz-result, get-recommendations) and 24 in route files. Ideally, the inline ones should be moved to controllers."
- **Follow-up:** "Why did you keep them inline?"

Q102. What's the difference between /api/user/register and /api/user/verify?

- **Short:** "Register creates temporary data (in JWT) and sends OTP. Verify validates the OTP and actually creates the user in the database. Two-step to ensure email is real."
- **Follow-up:** "What if register is called multiple times with same email?"

Q103. How does the recommendations API work?

- **Short:** "GET /api/get-recommendations/:userId — Fetches last 5 quiz results sorted by date. Calculates percentage of last quiz. If < 60%, suggests reviewing. If > 80%, suggests advancing. Otherwise, says 'keep practicing'."
- **Follow-up:** "Is this AI-powered?" → No, it's rule-based.

Q104. What happens if an unauthenticated user calls a protected API?

- **Short:** "isAuth middleware checks for token. If no token, returns 401 {message: 'Unauthorized. Please log in.'}. Frontend shows this as toast and redirects to login."
- **Follow-up:** "What if someone sends an expired token?"

Q105. How is FormData handled for file uploads?

- **Short:** “Frontend creates FormData object, appends fields + file. Sends with Axios POST (no Content-Type header needed — Axios auto-sets multipart/form-data). Backend Multer parses it, puts file in req.file, text fields in req.body.”
 - **Follow-up:** “Why can’t you send files as JSON?”
-

H. CHALLENGES — 10 Questions

Q106. What was the most challenging part?

- **Short:** See the natural answer in Part 10 — AI response parsing + rate limiting.
- **Follow-up:** “How would you solve it differently today?”

Q107. Did you face any authentication challenges?

- **Short:** “Yes — implementing OTP without database storage. Solved by embedding OTP in JWT. Also, Google OAuth required handling the case where a user might try to login with email/password after Google signup.”
- **Follow-up:** “How do you handle that edge case?”

Q108. What deployment challenges did you face?

- **Short:** “Render free tier blocks SMTP port 465, so OTP emails fail. Solved by logging OTP to console as fallback. Also, Render has ephemeral storage — uploaded files disappear on redeploy.”
- **Follow-up:** “How would you fix the email issue permanently?”

Q109. Did you face any CORS issues?

- **Short:** “Yes, initially requests from Vercel to Render were blocked. Solved by adding Vercel URL to allowed origins and configuring cors middleware. Currently using permissive config for ease.”
- **Follow-up:** “What’s the security risk?”

Q110. How did you handle the AI returning unpredictable responses?

- **Short:** “Regex cleaning of markdown fences, try-catch around JSON.parse, returning raw response if parsing fails, and fetchWithRetry for rate limits.”
- **Follow-up:** “What percentage of AI responses fail to parse?”

Q111. What database challenges did you face?

- **Short:** “Ensuring subscription array doesn’t have duplicates (check before push), cleaning up all related data on course deletion (cascading), and the inefficient stats query.”
- **Follow-up:** “How would you prevent subscription duplicates at DB level?”

Q112. Did you face any file upload challenges?

- **Short:** “Managing file names (solved with UUID), ensuring cleanup on deletion (solved with Promise.all + unlink), and video file sizes potentially being large.”
- **Follow-up:** “What’s the max upload size your server handles?”

Q113. How did you debug issues during development?

- **Short:** “Console.log for backend, browser DevTools Network tab for API calls, React DevTools for component state. Postman for testing APIs independently.”
- **Follow-up:** “Do you use any logging library?”

Q114. What security challenges did you encounter?

- **Short:** “Payment verification (solved with HMAC), protecting premium content server-side, storing secrets securely with .env, and ensuring OTP expiration.”
- **Follow-up:** “Is your current setup production-ready security-wise?”

Q115. How did you handle the Razorpay integration complexity?

- **Short:** “Followed Razorpay’s official documentation. Key insight: create order on server, open check-out on client, verify signature on server. The HMAC signature was the trickiest part.”
 - **Follow-up:** “What’s HMAC?”
-

I. FUTURE IMPROVEMENTS — 10 Questions

Q116. What would you improve about file storage?

- **Short:** “Migrate from local Multer uploads to AWS S3 or Cloudinary. Local storage doesn’t persist on Render, isn’t CDN-backed, and can’t scale.”
- **Follow-up:** “How would you implement S3 upload?”

Q117. How would you improve the admin stats query?

- **Short:** “Replace `(await User.find()).length` with `await User.countDocuments()`. The current approach loads every document into memory just to count — $O(n)$ memory vs $O(1)$ with `countDocuments`.”
- **Follow-up:** “What other queries need optimization?”

Q118. How would you add pagination?

- **Short:** “Use Mongoose `.skip()` and `.limit()` with page and limit query params. Example: `Courses.find().skip((page-1) * limit).limit(limit)`. Also return total count for frontend pagination UI.”
- **Follow-up:** “What’s cursor-based pagination?”

Q119. How would you improve the CORS configuration?

- **Short:** “Validate origin against the `allowedOrigins` array. Return `callback(null, true)` only for whitelisted origins. Different configs for development vs production.”
- **Follow-up:** “How would you implement this?”

Q120. How would you add input validation?

- **Short:** “Use Zod or Joi to create validation schemas. Validate `req.body` before controller logic runs. Return descriptive 400 errors for invalid input.”
- **Follow-up:** “What’s Zod?”

Q121. How would you add testing?

- **Short:** “Unit tests with Jest for utility functions (OTP generation, password hashing). Integration tests with Supertest for API endpoints. Frontend tests with React Testing Library.”
- **Follow-up:** “What would you test first?”

Q122. How would you fix the Google OAuth password issue?

- **Short:** “Make password optional in schema. Add an authProvider field ('local' or 'google'). Block email/password login for Google users. Show appropriate UI.”
- **Follow-up:** “What if a user registers with email first, then tries Google login?”

Q123. How would you add real-time features?

- **Short:** “Use Socket.IO for real-time progress updates, live quiz competitions, and admin notifications when a new user registers.”
- **Follow-up:** “What is WebSocket?”

Q124. How would you improve email reliability?

- **Short:** “Switch from Nodemailer + Gmail to a transactional email service like SendGrid, Resend, or AWS SES. More reliable, better deliverability, templates support.”
- **Follow-up:** “What’s a transactional email?”

Q125. How would you add caching?

- **Short:** “Redis cache for frequently accessed data like course list (GET /api/course/all). Cache invalidation on course creation/update/deletion.”
 - **Follow-up:** “What’s cache invalidation?”
-

J. TRICKY QUESTIONS — 20 Questions**Q126. Your CORS config allows all origins. Isn’t that a security risk?**

- **Short:** “Yes, any website could make requests to my API. In production, I should validate the origin against a whitelist. The allowedOrigins array exists in my code but isn’t being enforced.”
- **Follow-up:** “What attacks does CORS prevent?”

Q127. What happens if your JWT secret is leaked?

- **Short:** “Attacker could forge tokens and impersonate any user. Need to immediately rotate the secret. All existing tokens become invalid — users have to log in again.”
- **Follow-up:** “How do you rotate JWT secrets safely?”

Q128. Your OTP is Math.random(). Is that secure?

- **Short:** “No — Math.random() is not cryptographically secure. An attacker could theoretically predict the next OTP. Better to use crypto.randomInt(100000, 999999) from Node.js crypto module.”
- **Follow-up:** “What makes something cryptographically secure?”

Q129. Can a user buy the same course twice?

- **Short:** “No — the checkout function checks `user.subscription.includes(course._id)` and returns 400 ‘You already have this course’ if true.”
- **Follow-up:** “What about race conditions with simultaneous requests?”

Q130. What happens if the database goes down during payment verification?

- **Short:** “Payment might be charged on Razorpay but the user won’t be enrolled. There’s no retry or webhook fallback. Should implement Razorpay webhooks for reliable payment confirmation.”
- **Follow-up:** “What are webhooks?”

Q131. Your token is in `req.headers.token`, not `Authorization`. Why?

- **Short:** “It’s a custom implementation. Standard is `Authorization: Bearer <token>`. My approach works but isn’t REST standard. Some libraries and tools expect the `Authorization` header.”
- **Follow-up:** “How would you change it?”

Q132. What if two users register with the same email simultaneously?

- **Short:** “The email field has `unique: true` index. The second `User.create()` would throw a MongoDB duplicate key error. The `TryCatch` wrapper catches it and returns 500.”
- **Follow-up:** “Should the error message be more specific?”

Q133. Your video files are served as static files. Can anyone access them?

- **Short:** “Yes — if someone knows the filename, they can access it at `/uploads/filename` without authentication. The authentication check only happens at the lecture API endpoint, not at the file level.”
- **Follow-up:** “How would you secure static file access?”

Q134. What if `Promise.all` fails during course deletion?

- **Short:** “If one file deletion fails, `Promise.all` rejects and the whole operation stops. DB records might not be deleted. Should use `Promise.allSettled` instead, or delete files after DB cleanup.”
- **Follow-up:** “What’s the difference between `Promise.all` and `Promise.allSettled`?”

Q135. Why does `getAllStats` not use `countDocuments()`?

- **Short:** “That’s a code oversight. Current code: `(await User.find()).length` loads all documents into memory just to count them. `User.countDocuments()` does the counting in MongoDB engine — much more efficient.”
- **Follow-up:** “How much memory difference at 10,000 users?”

Q136. Can an admin see other users’ passwords?

- **Short:** “No — passwords are hashed with `bcrypt`. Even in the database, only hashed values are stored. The `getAllUser` query uses `.select('-password')` to exclude passwords from the response.”
- **Follow-up:** “What does `.select('-password')` do?”

Q137. What if the Gemini API key is exposed in the client?

- **Short:** “The Gemini API key is only on the server (`process.env.GEMINI_API_KEY`). Frontend never sees it. Requests go through my backend which proxies to Gemini. This is the correct pattern.”
- **Follow-up:** “What if it was in the frontend?”

Q138. How do you handle token expiration on the frontend?

- **Short:** "When fetchUser (GET /api/user/me) returns 401, the frontend sets isAuthenticated = false and redirects to login. The token expires after 15 days."
- **Follow-up:** "How would you implement token refresh?"

Q139. Your user schema has subscription as an array. What if a user buys 100 courses?

- **Short:** "MongoDB documents have a 16MB limit. An array of 100 ObjectIds is tiny (~2.4KB). But for very large numbers, a separate Enrollment collection would be better."
- **Follow-up:** "When would you switch to a separate collection?"

Q140. Is your API RESTful?

- **Short:** "Mostly. It uses proper HTTP methods and URL patterns. But some deviations: POST /api/user/progress uses query params instead of request body, some routes are inconsistent (checkout vs verification naming)."
- **Follow-up:** "What are the REST principles?"

Q141. What if someone repeatedly calls the quiz generation API?

- **Short:** "Currently no rate limiting on my server. They could exhaust my Gemini API quota. Should implement express-rate-limit middleware."
- **Follow-up:** "How does rate limiting work?"

Q142. Your app has no tests. How do you ensure quality?

- **Short:** "Currently manual testing. This is a limitation. I'd add Jest for unit tests (especially for auth logic and payment verification) and Supertest for integration tests."
- **Follow-up:** "What would you test first if you had to pick one function?"

Q143. What happens when Render's free tier spins down?

- **Short:** "Render free tier sleeps after 15 minutes of inactivity. First request after sleep takes 30-60 seconds to cold start. Also, any files in uploads/ are lost on restart."
- **Follow-up:** "How would you solve the cold start problem?"

Q144. Can a regular admin delete another admin's account?

- **Short:** "The deleteUser function in admin controller doesn't check this — any admin can delete any user. Should add protection so admins can't delete other admins or superadmins."
- **Follow-up:** "How would you implement this protection?"

Q145. Why do you have both role and mainrole fields?

- **Short:** "mainrole is immutable (set at creation, e.g., 'superadmin'). role is mutable (can be toggled between 'user' and 'admin'). This prevents a promoted admin from accidentally losing their special status if their role is toggled."
- **Follow-up:** "What if you need more than 3 roles?"